RetrieveGPT: Merging Prompts and Mathematical Models for Enhanced Code-Mixed Information Retrieval

Aniket Deroy[1,*,†], Subhankar Maity[1]

[1]IIT Kharagpur, Kharagpur, India

Abstract

Code-mixing, the integration of lexical and grammatical elements from multiple languages within a single sentence, is a widespread linguistic phenomenon, particularly prevalent in multilingual societies. In India, social media users frequently engage in code-mixed conversations using the Roman script, especially among migrant communities who form online groups to share relevant local information. This paper focuses on the challenges of extracting relevant information from code-mixed conversations, specifically within Roman transliterated Bengali mixed with English. This study presents a novel approach to address these challenges by developing a mechanism to automatically identify the most relevant answers from code-mixed conversations. We have experimented with a dataset comprising of queries and documents from Facebook, and Query Relevance files (QRels) to aid in this task. Our results demonstrate the effectiveness of our approach in extracting pertinent information from complex, code-mixed digital conversations, contributing to the broader field of natural language processing in multilingual and informal text environments. We use GPT-3.5 Turbo via prompting alongwith using the sequential nature of relevant documents to frame a mathematical model which helps to detect relevant documents corresponding to a query.

Keywords

GPT, Relevance, Code Mixing, Probability, Prompt Engineering

arXiv:2411.04752v3 [cs.CL] 26 Mar 2025

1. Introduction

Code-mixing, where elements from multiple languages are blended within a single sentence, is a natural and widespread phenomenon in multilingual societies [1, 2]. Code mixing is a global phenomenon where speakers often switch between languages depending on context, audience, and medium of communication [3]. With the rapid rise of online social networking, this practice has become increasingly common in digital conversations, where users frequently combine their native languages with others, often using foreign scripts [4].

One notable trend in India is the use of the Roman script to communicate in native languages on social media platforms [5]. This practice is especially common among migrant communities who form online groups to share information and experiences relevant to their unique circumstances [6]. For instance, Bengali speakers from West Bengal who have migrated to urban centers like Delhi or Bangalore often establish groups such as "Bengali in Delhi" on platforms like Facebook and WhatsApp. These groups serve as vital hubs for exchanging advice on a wide range of local issues, from housing and employment to navigating new social environments.

The COVID-19 pandemic highlighted the importance of these online communities as critical sources of information [7]. During this period, these groups became essential for sharing experiences, seeking support, and keeping up with the frequently changing government guidelines. However, the informal and often colloquial nature of the language used in these code-mixed conversations, typically transliterated into Roman script, presents significant challenges for information retrieval. The lack of standardization, combined with the blending of languages, makes it difficult to identify and extract relevant answers, especially for those who might seek similar information at a later time [8].

Forum for Information Retrieval Evaluation, December 12-15, 2024, India

*Corresponding author.

roydanik18@kgpian.iitkgp.ac.in (A. Deroy); subhankar.ai@kgpian.iitkgp.ac.in (S. Maity)

0000-0001-7190-5040 (A. Deroy); 0009-0001-1358-9534 (S. Maity)

© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

This paper addresses the challenge of extracting relevant information from code-mixed digital conversations, with a specific focus on Roman transliterated Bengali mixed with English. While code-mixing is a well-recognized phenomenon in natural language processing (NLP), the unique characteristics of transliterated text—such as variations in spelling, grammar, and syntax—complicate the task of effective information retrieval [9]. To tackle this issue, we have developed a mechanism that identifies the most relevant answers from these complex, multilingual discussions.

We begin experimenting with a dataset of code-mixed conversations collected from Facebook, which has been carefully annotated to reflect query relevance (QRels). This dataset forms the basis of our study and is crucial for evaluating the effectiveness of our approach.

We leverage GPT-3.5 Turbo [10] by employing carefully designed prompts that guide the model to evaluate the relevance of documents with respect to a given query. This involves not only the semantic understanding capabilities of GPT-3.5 Turbo but also the strategic use of the sequential nature of documents. Often, documents are part of a series or a conversation where the relevance to a query can be influenced by preceding or succeeding documents. By acknowledging this sequence, we can better capture contextual relationships that might be missed if documents were considered in isolation.

To formalize this process, we integrate GPT-3.5 Turbo's outputs into a mathematical model. This model takes into account the sequential dependencies among documents, treating the task of relevance detection as a problem of finding the optimal path or chain of relevance across the sequence.

2. Related Work

Code-mixing and transliteration have gained increasing attention in the field of natural language processing (NLP), especially as global communication becomes more digital and multilingual [11, 12, 13]. This section reviews key studies related to code-mixing, information retrieval from code-mixed text, and the challenges of processing Roman transliterated languages, particularly in the context of Indian languages. Code-mixing, where speakers blend elements from multiple languages within a single utterance, is a common linguistic phenomenon in multilingual societies [13]. Early studies on code-mixing focused primarily on sociolinguistic aspects, examining how and why speakers switch languages within conversations [11, 12, 13]. However, with the advent of digital communication, researchers have increasingly turned their attention to computational methods for processing and understanding code-mixed text [14].

Several studies have explored various NLP tasks, such as part-of-speech tagging, language identification, and sentiment analysis, in code-mixed settings [15]. [16] provided one of the earliest comprehensive analyses of code-mixed text, highlighting the unique challenges it poses for traditional NLP pipelines, such as non-standard spelling, syntax variations, and the blending of multiple languages within a single text. More recent work by [17] introduced a code-mixed dataset, spanning multiple Indian languages, which has become a benchmark for evaluating NLP models in this domain.

Information retrieval (IR) in code-mixed settings is relatively underexplored compared to other NLP tasks [18]. However, the need for effective IR systems that can handle multilingual and code-mixed queries has become increasingly important, particularly in the context of digital information exchange on social media platforms. [19] investigated the problem of query-focused summarization in code-mixed social media data, emphasizing the complexity of extracting relevant information from noisy, informal text. Work by [20] addressed code-mixed question answering, where the goal is to identify correct responses from a mixed-language corpus. Their approach involved using translation models to standardize the text before applying traditional IR techniques, demonstrating that even simple translation-based methods can significantly improve performance. However, these methods often fail to capture the nuances of code-mixed language, such as cultural context and colloquial expressions.

Roman script transliteration of Indian languages, commonly referred to as "Romanagari" [21] for languages like Hindi, is a widespread practice in digital communication. Transliteration introduces additional challenges for NLP, as it often involves non-standard spellings and inconsistent usage. For instance, multiple transliterations may exist for the same word, depending on the speaker's regional

accent, literacy in the original script, or personal preference.

Notable efforts in this area include the work by [22], which explored transliteration normalization for Hindi-English code-mixed text. They developed algorithms to map Romanized text back to its original script, enabling more accurate processing by traditional NLP models. However, normalization remains a challenging task due to the inherent variability in transliterated text. In the context of Bengali, the Roman script transliteration is less standardized than for Hindi, leading to even greater variability in spelling and grammar. [23] addressed this issue by creating a Roman Bengali dataset and proposed methods for transliteration normalization and language identification. Their work highlights the difficulties of processing Roman Bengali and the need for specialized approaches tailored to the characteristics of the language.

While these studies provide valuable insights into code-mixing, transliteration, and information retrieval, there is a noticeable gap in addressing the specific challenges of extracting relevant information from code-mixed conversations in Roman transliterated Bengali. Our work builds on the foundations laid by previous research but focuses on the unique intersection of these challenges in a real-world context. By developing a mechanism to identify relevant answers in code-mixed discussions, we aim to contribute to the growing body of research on multilingual NLP and enhance the accessibility of information in linguistically diverse online communities.

Large Language Models (LLMs) [24, 25, 26] like GPT-3 have shown promise in various NLP tasks, including LI. Previous works have demonstrated the capability of GPT-3 in performing zero-shot and few-shot learning, making it a potentially powerful tool for LI in resource-constrained settings. However, the application of LLMs [27, 28, 29, 30] to code-mixed and morphologically rich languages remains underexplored. Recent studies, have started to explore the use of transformers and pre-trained models for multilingual LI, but the effectiveness of these models in Bengali languages requires further investigation.

This section places our work within the context of existing research, highlighting the contributions of prior studies while identifying gaps that our research aims to fill.

3. Dataset

This shared task consists of a single dataset [31] for code mixed information retrieval. The corpus consists of 107900 documents in the training set and 20 queries in the training set. There are 30 queries in the testing set. The dataset is in roman transliterated bengali mixed with english language.

4. Task Definition

The task [1] is to automatically determine the relevance of a query to a document within code-mixed data, specifically focusing on English and Roman transliterated Bengali.

Given a query and a document, the goal is to classify whether the query is relevant or not relevant to the document. Based on the relevance we have to rank the documents. This involves handling the complexities of code-mixing, where elements from both languages are used within the same text, and dealing with the informal and non-standardized nature of the language. The system must accurately capture the semantic relationship between the query and the document despite these linguistic challenges.

5. Methodology

5.1. Why Prompting?

Prompting [32] for Information Retrieval is a burgeoning approach that leverages large language models (LLMs) to enhance the retrieval of relevant information from complex, unstructured data, such as

[1] https://cmir-iitbhu.github.io/cmir/

code-mixed text or informal online conversations [32]. Below are several reasons why prompting is becoming an effective strategy in information retrieval (IR):

- **Handling Ambiguity and Contextual Nuances:** Traditional IR systems often struggle with understanding the nuanced language, ambiguity, and context found in unstructured or informal text, such as code-mixed conversations. Prompting LLMs allows these models to interpret context more effectively by guiding them to generate or rank responses that are contextually appropriate, even when dealing with code-mixing or informal language structures [33]. By crafting specific prompts, users can elicit more relevant and accurate results that account for the complexities of the input text.
- **Enhanced Language Understanding:** Large language models like GPT-3.5 are pre-trained on vast datasets that include a variety of languages and dialects [34]. This extensive training enables them to understand and generate text across different languages and contexts [34]. By using prompting, these models can be directed to focus on the most relevant aspects of a query or document, improving the retrieval process even in multilingual and code-mixed scenarios. For example, when retrieving information from Roman transliterated Bengali mixed with English, an LLM can be prompted to recognize and process the code-mixed language more effectively than traditional IR systems.
- **Adaptability to Informal and Unstructured Text:** Prompting allows LLMs to adapt to the informal and often unstructured nature of social media text [35], which is common in online communities. This flexibility is particularly beneficial when dealing with code-mixed or transliterated text, where the lack of standardization poses a challenge to conventional IR techniques. Prompted language models can generate or filter responses that align more closely with the informal tone and style of the original text, thereby improving the relevance of the retrieved information.
- **Reduction of Noise and Irrelevance:** One of the major challenges in IR is filtering out irrelevant or noisy data, especially in informal online conversations where off-topic or redundant information is common. By using targeted prompts, LLMs can be instructed to prioritize certain types of information, such as direct answers to specific questions, while de-emphasizing or ignoring irrelevant content [36]. This leads to a more efficient and effective retrieval process, particularly in environments where users are seeking specific answers within a sea of mixed and informal language.
- **Scalability and Customization:** Prompting for information retrieval offers scalability and customization that traditional IR systems might lack. By designing prompts tailored to specific contexts or types of queries, LLMs can be dynamically adjusted to meet the needs of different retrieval tasks [36]. This customization is particularly useful in handling domain-specific language or code-mixed scenarios, where standard IR systems might require extensive re-training or re-configuration.
- **Real-Time Processing and Interaction:** In real-time communication platforms, the ability to quickly retrieve relevant information based on ongoing conversations is crucial. Prompting enables LLMs to process and respond to queries in real-time, enhancing the interactivity and responsiveness of the IR system [36]. This is especially beneficial in scenarios where users are engaged in active discussions and require immediate, contextually relevant information.

5.2. Merging Prompt and Mathematical Model-Based Approaches

We used the GPT-3.5 Turbo model via prompting through the OpenAI API[2] to solve the document retrieval task. The process begins by first converting all the code-mixed sentences to english for both the queries and the documents. After this we try to determine the relevance scores, we used the following prompt:

"*Given the query <query> and the document <document>, find how relevant is the query to the document based on semantic similarity. Provide a relevance score between 0 and 1. Only state the score.*"

[2]https://platform.openai.com/docs/models/gpt-3-5-turbo

After the prompt is provided to the LLM, the following steps happen internal to the LLM while generating the output. The following outlines the steps that occur internally within the LLM, summarizing the prompting approach using GPT-3.5 Turbo:

Step 1: Tokenization

- **Prompt:** $X = [x_1, x_2, \ldots, x_n]$
- The input text (prompt) is first tokenized into smaller units called tokens. These tokens are often subwords or characters, depending on the model's design.
- **Tokenized Input:** $T = [t_1, t_2, \ldots, t_m]$

Step 2: Embedding

- Each token is converted into a high-dimensional vector (embedding) using an embedding matrix E.
- **Embedding Matrix:** $E \in \mathbb{R}^{|V| \times d}$, where $|V|$ is the size of the vocabulary and d is the embedding dimension.
- **Embedded Tokens:** $T_{\text{emb}} = [E(t_1), E(t_2), \ldots, E(t_m)]$

Step 3: Positional Encoding

- Since the model processes sequences, it adds positional information to the embeddings to capture the order of tokens.
- **Positional Encoding:** $P(t_i)$
- **Input to the Model:** $Z = T_{\text{emb}} + P$

Step 4: Attention Mechanism (Transformer Architecture)

- **Attention Score Calculation:** The model computes attention scores to determine the importance of each token relative to others in the sequence.
- **Attention Formula:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

- where Q (query), K (key), and V (value) are linear transformations of the input Z.
- This attention mechanism is applied multiple times through multi-head attention, allowing the model to focus on different parts of the sequence simultaneously.

Step 5: Feedforward Neural Networks

- The output of the attention mechanism is passed through feedforward neural networks, which apply non-linear transformations.
- **Feedforward Layer:**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

- where W_1, W_2 are weight matrices and b_1, b_2 are biases.

Step 6: Stacking Layers

- Multiple layers of attention and feedforward networks are stacked, each with its own set of parameters. This forms the "deep" in deep learning.

- **Layer Output:**

$$H^{(l)} = \text{LayerNorm}(Z^{(l)} + \text{Attention}(Q^{(l)}, K^{(l)}, V^{(l)})) \tag{3}$$

$$Z^{(l+1)} = \text{LayerNorm}(H^{(l)} + \text{FFN}(H^{(l)})) \tag{4}$$

Step 7: Output Generation

- The final output of the stacked layers is a sequence of vectors.
- These vectors are projected back into the token space using a softmax layer to predict the next token or word in the sequence.
- **Softmax Function:**

$$P(y_i|X) = \frac{\exp(Z_i)}{\sum_{j=1}^{|V|} \exp(Z_j)} \tag{5}$$

- where Z_i is the logit corresponding to token i in the vocabulary.
- The model generates the next token in the sequence based on the probability distribution, and the process repeats until the end of the output sequence is reached.

Step 8: Decoding

- The predicted tokens are then decoded back into text, forming the final output.
- **Output Text:** $Y = [y_1, y_2, \ldots, y_k]$

After obtaining the relevance score, we used the following mathematical formulation to account for the sequential presence of relevant documents. This can be written as follows:

$$P(D_{n+1} \mid D_n) = \begin{cases} \text{Score}(D_{n+1}) & \text{if Score}(D_{n+1}) < 0.3, D_n = relevant \\ \text{Score}(D_{n+1}) & \text{if } n = -1 \\ 0.2 + \text{Score}(D_{n+1}) & \text{if Score}(D_{n+1}) >= 0.3, D_n = relevant \\ \text{Score}(D_{n+1}) & otherwise \end{cases}$$

This equation now reflects that if the score of the current document D_n is less than 0.3 and the previous document is relevant, the probability of the current document being relevant is simply equal to the relevance score of current document.

If the previous document is relevant and if the score of the current document D_n is greater than equal to 0.3 then the probability that the current document is relevant is 0.2 + Score for the current document. For the first document, the probability is equal to the relevance score of current document. In all other situations, the probability is equal to the relevance score of current document. If the probability score of a particular document is greater than 0.5, we consider the document to be relevant to the query. Like this we found out all documents which are relevant to a query.

An example of the mathematical formulation and how it helps to detect relevant documents is shown in Table 1. The table reflects a range of documents to a code mixed query. The relevance scores help identify how closely each document addresses the query, while the probability scores provide insight into the potential usefulness of the documents based on the provided content. Overall, Documents 1-4 stands out as particularly relevant based on probability scores.

For the five results reported, we ran the GPT model at different temperature values namely 0.5, 0,6, 0.7, 0.8, and 0.9. The diagram for GPT-3.5 Turbo is shown in Figure 1. The figure representing the methodology is shown in Figure 2.

At lower temperatures, the model's responses are more deterministic and focused. It generates outputs that are likely to be relevant and closely aligned with the input, making it useful for tasks requiring precision, such as retrieving specific information or handling queries with clear intent. At higher temperatures results in highly diverse and less predictable outputs. It can be useful in exploratory tasks where creativity and variation are needed, but it may also risk generating less coherent or relevant responses. In code-mixed scenarios, this could capture the full spectrum of linguistic creativity but might require careful handling to ensure relevance. So we used a temperature range of 0.5 to 0.9.

Query	Document	Relevance Score	Probability Score
Kivabe bhalo bhabe ingreji shikhbo?	Shudhumatro lekhar opor focus korle hobe na. Ingreji songs shunle shikha sohoj hoy.	0.55	0.55 (1)
Kivabe bhalo bhabe ingreji shikhbo?	Ingreji film dekhle vocabulary bere jaye. Conversation practice korao khub helpful.	0.45	0.65 (1)
Kivabe bhalo bhabe ingreji shikhbo?	Ingreji shikhar jonyo grammar book khub important. Kintu speaking practice beshi dorkar.	0.35	0.55 (1)
Kivabe bhalo bhabe ingreji shikhbo?	Bhasha shikhte bhalo tutor nirbachon kora joruri. Bibhinno apps byabohar kore practice kora jaye.	0.45	0.65 (1)
Kivabe bhalo bhabe ingreji shikhbo?	time lagbe.	0.20	0.20 (0)

Table 1
Example of Query and Document Relevance along with probability scores and relevance scores. Beside the probability score the relevance of a document to a query is stated in braces. 1 represents relevant and 0 represents not-relevant.

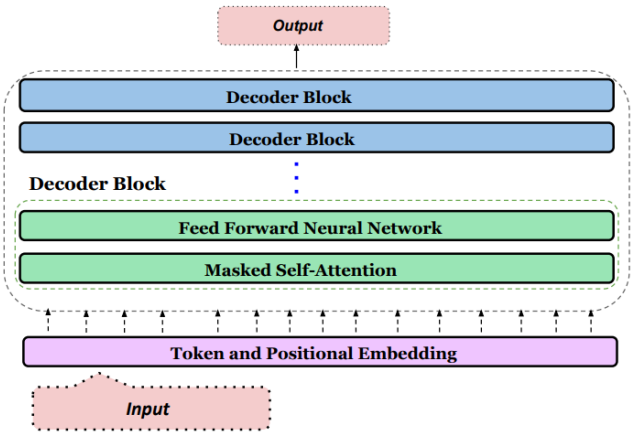

Figure 1: An overview of the GPT-3.5 Turbo architecture.

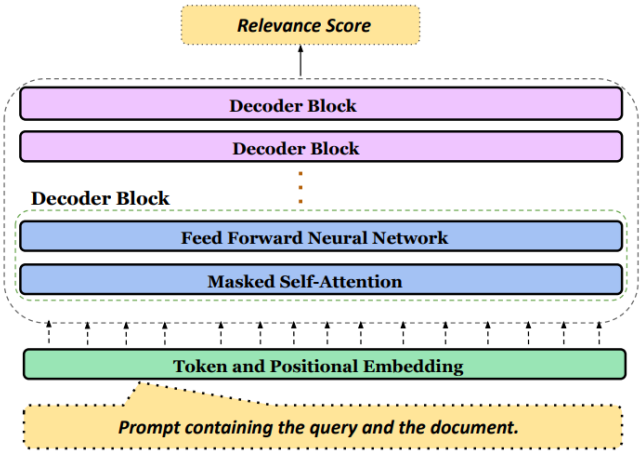

Figure 2: Overview diagram of the methodology followed for GPT-3.5 Turbo.

MAP Score	ndcg Score	p@5 Score	p@10 Score	Team Name	Submission File	Rank
0.701773	0.797937	0.793333	0.766667	TextTitans	submit_cmir	5
0.701773	0.797937	0.793333	0.766667	TextTitans	submit_cmir_1	4
0.701773	0.797937	0.793333	0.766667	TextTitans	submit_cmir_2	3
0.701773	0.797937	0.793333	0.766667	TextTitans	submit_cmir_3	2
0.703734	0.799196	0.793333	0.766667	TextTitans	submit_cmir_4	1

Table 2
A Comparison of MAP, NDCG, P@5, and P@10 Scores for the TextTitans Team.

6. Results

Table 2 presents the evaluation metrics for different submissions [37] for the team named "TextTitans". The metrics used to assess the performance are MAP Score, ndcg Score, p@5 Score, and p@10 Score. Here's what these results imply. MAP is a common metric in information retrieval that measures the precision of results across multiple queries. A higher MAP score indicates that relevant documents are consistently ranked higher across all queries. In the table, the MAP scores for the first four submissions are identical (0.701773), while the fifth submission slightly improves to 0.703734. This indicates that the fifth submission is marginally better in terms of ranking relevant results across multiple queries. The ndcg score measures the quality of the ranking based on the position of relevant documents. A higher ndcg score suggests that relevant documents are placed higher in the ranking. The scores are also very similar across submissions, with the first four submissions having an ndcg score of 0.797937, and the fifth submission showing a slight improvement to 0.799196. This suggests a minor improvement in ranking relevant documents for the fifth submission. p@5 measures how many of the top 5 ranked documents are relevant. A score of 1 would mean that all 5 of the top-ranked documents are relevant. All submissions have the same p@5 score of 0.793333, indicating that the top 5 results are equally accurate across all submissions. Precision@10 measures how many of the top 10 ranked documents are relevant. Like p@5, a higher score is better. Similar to p@5, all submissions have the same p@10 score of 0.766667, showing no variation in the top 10 results across the different submissions. The metrics are very consistent across all submissions, with only minor improvements in MAP and NDCG scores for the fifth submission. The fifth submission shows a slight improvement in ranking and retrieval performance, but the changes are minimal. The p@5 and p@10 scores indicate that the precision of the top 5 and top 10 results is identical across all submissions, suggesting that the models are performing similarly in identifying the most relevant documents. Overall, while there is a slight improvement in the last submission, the models generally perform similarly across all metrics.

7. Conclusion

In conclusion, this study addresses the critical challenges of extracting relevant information from code-mixed conversations, specifically within Roman transliterated Bengali mixed with English. This linguistic phenomenon is prevalent among migrant communities in India, who often rely on social media platforms to share and seek vital information, especially during crises like the COVID-19 pandemic. The informal and non-standardized nature of these conversations presents unique difficulties for information retrieval. To tackle these challenges, we developed a novel approach that leverages the GPT-3.5 Turbo model in conjunction with a sequential engineering approach, achieving notable success in retrieving pertinent answers from complex, code-mixed digital conversations. The effectiveness of our method is demonstrated through the results on the test set documents and queries, which provides a valuable resource for future research in natural language processing within multilingual and informal text environments. This work contributes to enhancing information accessibility for marginalized

communities, underscoring the potential of advanced AI models in bridging communication gaps in diverse linguistic landscapes. We observe that the GPT-3.5 model along with mathematical formulation approach performs well for the task of Code mixed information retrieval, though there is scope for improvement.

References

[1] E. Sippola, Multilingualism and the structure of code-mixing, in: The Routledge handbook of Pidgin and Creole languages, Routledge, 2020, pp. 474–489.

[2] E. O. Aboh, Lessons from neuro-(a)-typical brains: universal multilingualism, code-mixing, recombination, and executive functions, Frontiers in psychology 11 (2020) 488.

[3] A. De Swaan, Words of the world: The global language system, John Wiley & Sons, 2013.

[4] C. Lee, Multilingual resources and practices in digital communication, in: The Routledge handbook of language and digital communication, Routledge, 2015, pp. 118–132.

[5] S. Shekhar, H. Garg, R. Agrawal, S. Shivani, B. Sharma, Hatred and trolling detection transliteration framework using hierarchical lstm in code-mixed social media text, Complex & Intelligent Systems 9 (2023) 2813–2826.

[6] L. Komito, Social media and migration: Virtual community 2.0, Journal of the American society for information science and technology 62 (2011) 1075–1086.

[7] M. M. Meurer, M. Waldkirch, P. K. Schou, E. L. Bucher, K. Burmeister-Lamp, Digital affordances: How entrepreneurs access support in online communities during the covid-19 pandemic, Small Business Economics (2022) 1–27.

[8] D. D. Lewis, K. S. Jones, Natural language processing for information retrieval, Communications of the ACM 39 (1996) 92–101.

[9] M. Janse, N. Vassalou, D. Papazachariou, Variation in the vowel system of mišótika cappadocian: Findings from two refugee villages in greec, in: 13th International Conference on Greek Linguistics, University of Westminster, 2017.

[10] T. B. Brown, Language models are few-shot learners, arXiv preprint ArXiv:2005.14165 (2020).

[11] T. Jauhiainen, H. Jauhiainen, K. Linden, A survey on automatic language identification in written texts, in: Journal of Artificial Intelligence Research, volume 65, 2019, pp. 675–782.

[12] Y. Muthusamy, R. A. Cole, B. T. Oshika, Automatic language identification: A review/tutorial, in: IEEE Signal Processing Magazine, volume 11, 1994, pp. 33–41.

[13] G. I. Ahmad, J. Singla, Sentiment analysis of code-mixed social media text (sa-cmsmt) in indian-languages, in: 2021 International Conference on Computing Sciences (ICCS), IEEE, 2021, pp. 25–33.

[14] A. F. Hidayatullah, A. Qazi, D. T. C. Lai, R. A. Apong, A systematic review on language identification of code-mixed text: techniques, data availability, challenges, and framework development, IEEE access 10 (2022) 122812–122831.

[15] G. I. Ahmad, J. Singla, A. Anis, A. A. Reshi, A. A. Salameh, Machine learning techniques for sentiment analysis of code-mixed and switched indian social media text corpus: A comprehensive review, International Journal of Advanced Computer Science and Applications 13 (2022).

[16] A. F. Hidayatullah, A. Qazi, D. T. C. Lai, R. A. Apong, A systematic review on language identification of code-mixed text: techniques, data availability, challenges, and framework development, IEEE access 10 (2022) 122812–122831.

[17] A. Pratapa, G. Bhat, M. Choudhury, S. Sitaram, S. Dandapat, K. Bali, Language modeling for code-mixing: The role of linguistic theory based synthetic data, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 1543–1553.

[18] U. Barman, Automatic processing of code-mixed social media content, Ph.D. thesis, Dublin City University, 2019.

[19] D. Gupta, A. Ekbal, P. Bhattacharyya, A semi-supervised approach to generate the code-mixed

text using pre-trained encoder and transfer learning, in: T. Cohn, Y. He, Y. Liu (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 2267–2280. URL: https://aclanthology.org/2020.findings-emnlp.206. doi:10.18653/v1/2020.findings-emnlp.206.

[20] K. R. Chandu, A. W. Black, Style variation as a vantage point for code-switching, arXiv preprint arXiv:2005.00458 (2020).

[21] R. Mhaiskar, Romanagari an alternative for modern media writings, Bulletin of the Deccan College Research Institute 75 (2015) 195–202.

[22] K. Bali, J. Sharma, M. Choudhury, Y. Vyas, "i am borrowing ya mixing?" an analysis of english-hindi code mixing in facebook, in: Proceedings of the first workshop on computational approaches to code switching, 2014, pp. 116–126.

[23] B. Sarkar, N. Sinhababu, M. Roy, P. K. D. Pramanik, P. Choudhury, Mining multilingual and multiscript twitter data: unleashing the language and script barrier, International Journal of Business Intelligence and Data Mining 16 (2020) 107–127.

[24] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, Language models are unsupervised multitask learners, in: OpenAI Blog, volume 1, 2019.

[25] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, Journal of Machine Learning Research 21 (2020) 1–67.

[26] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, in: arXiv preprint arXiv:1907.11692, 2019.

[27] W. X. Zhao, K. Zhou, J. Li, X. Tang, J. J. Wang, J. Liu, T. Wang, Y. Bao, J.-R. Wen, A survey of large language models, in: arXiv preprint arXiv:2303.18223, 2023.

[28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017) 5998–6008.

[29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, Fine-tuning gpt-2 for human-like text generation, in: arXiv preprint arXiv:1907.11692, 2019.

[30] R. Zellers, A. Holtzman, H. Rashkin, Y. Bisk, A. Farhadi, F. Roesner, Y. Choi, Defending against neural fake news, in: Advances in Neural Information Processing Systems, volume 32, 2019, pp. 9054–9065.

[31] S. Chanda, S. Pal, The effect of stopword removal on information retrieval for code-mixed data obtained via social media, SN Comput. Sci. 4 (2023) 494. URL: https://doi.org/10.1007/s42979-023-01942-7. doi:10.1007/S42979-023-01942-7.

[32] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, ACM Computing Surveys 55 (2023) 1–35.

[33] P. Singh, M. Patidar, L. Vig, Translating across cultures: Llms for intralingual cultural adaptation, arXiv preprint arXiv:2406.14504 (2024).

[34] G. Yenduri, M. Ramalingam, G. C. Selvi, Y. Supriya, G. Srivastava, P. K. R. Maddikunta, G. D. Raj, R. H. Jhaveri, B. Prabadevi, W. Wang, et al., Gpt (generative pre-trained transformer)–a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions, IEEE Access (2024).

[35] G. E. Zgheib, N. Dabbagh, Social media learning activities (smla): Implications for design., Online Learning 24 (2020) 50–66.

[36] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, R. McHardy, Challenges and applications of large language models, arXiv preprint arXiv:2307.10169 (2023).

[37] S. Chanda, S. Pal, Overview of the shared task on code-mixed information retrieval from social media data, in: Forum of Information Retrieval and Evaluation FIRE-2024, 2024.